

---

Seminar Report

Debugging: Methodology instead of Confusion

---

Fallak Amin, 598871  
Computer Science

# A Review of Software Logging Practices and Challenges

Based primarily on Batoun et al. (2024)

# Contents

|     |                                                |    |
|-----|------------------------------------------------|----|
| 1   | Introduction . . . . .                         | 3  |
| 2   | The Logging Pipeline . . . . .                 | 4  |
| 2.1 | Instrumentation . . . . .                      | 5  |
| 2.2 | Verbosity Levels and Configuration . . . . .   | 5  |
| 2.3 | Storage . . . . .                              | 5  |
| 2.4 | Analysis . . . . .                             | 6  |
| 3   | Research Focus in the Literature . . . . .     | 6  |
| 4   | Developer Practice on Stack Overflow . . . . . | 7  |
| 5   | Research Gap . . . . .                         | 8  |
| 6   | Critical Reflection . . . . .                  | 8  |
| 7   | Implications . . . . .                         | 9  |
| 7.1 | Implications for Developers . . . . .          | 9  |
| 7.2 | Implications for Researchers . . . . .         | 9  |
| 8   | Conclusion . . . . .                           | 9  |
|     | Bibliography . . . . .                         | 10 |

# 1 Introduction

Logging is necessary to examine and debug hard-to-understand failures. They can capture the runtime behavior of a system, which is why they are so important for developers. This does not mean that every log is instantly useful. Instead, only good logs can support the debugging and diagnosis of errors. In principle, logs can make invisible runtime behavior visible. In practice, however, logs only help when they contain the right information at the right level of detail.

A simple example illustrates the problem. Consider the following Java code, which reads an environment name from a text file and then uses it to select the corresponding database configuration.

```
String env = Files.readString(Path.of("env.txt"));

Map<String, String> configs = Map.of(
    "dev", "dev-db.local",
    "prod", "prod-db.company.com"
);

if (!configs.containsKey(env)) {
    throw new RuntimeException("Database config not found");
}
```

At first glance, the code looks correct. The file contains the value `prod`, and the configuration map also contains the key `prod`. Nonetheless, the program fails with the following output:

```
Exception in thread "main"
java.lang.RuntimeException: Database config not found
```

This failure shows why logging has to provide context. Adding only a generic message such as the following does not explain the actual problem:

```
System.out.println("Loading database config...");

if (!configs.containsKey(env)) {
    System.out.println("Database config missing.");
    throw new RuntimeException("Database config not found");
}
```

The resulting output confirms that the configuration is missing, but it still does not reveal why the lookup failed:

```
Loading database config...
```

```
Database config missing.  
Exception in thread "main" java.lang.RuntimeException:  
Database config not found
```

More useful logging exposes the relevant runtime context:

```
System.out.println("Requested environment: '" + env + "'");  
System.out.println("Environment length: " + env.length());  
System.out.println("Available configs: " + configs.keySet());
```

This produces information that directly explains the bug:

```
Requested environment: 'prod '  
Environment length: 5  
Available configs: [prod, dev]  
No config found for requested environment.
```

The problem is the trailing whitespace in `prod` . The map contains `prod`, but the program searches for `prod` with an additional character. A simple fix is to normalize the input, for example by using `strip()` after reading the file. The example demonstrates that logging is not merely about adding more output. Useful logs require design decisions and must reveal the specific runtime context needed to understand a failure.

Batoun et al. [1] address this broader view of logging by investigating logging practices from the creation of log statements to storage and analysis. Their work is relevant for a seminar on debugging because logging is one of the most common techniques used to understand failures in running systems. The authors conduct a systematic literature review of 204 research papers and complement it with an analysis of StackOverflow questions in order to identify the gap between academic research and the practical logging problems faced by developers. This report summarizes the paper, explains its main concepts, and critically reflects on its significance for debugging practice. The source code for the examples used in this report is available in the following GitHub repository: [https://github.com/F-A0/Seminar\\_Logging](https://github.com/F-A0/Seminar_Logging).

## 2 The Logging Pipeline

Software logging can be understood as a pipeline. First, the system is instrumented, meaning that developers or tools add log statements and configure which statements are shown. Second, the produced log data must be stored, protected, and managed. Third, the logs are analyzed so that developers and operators can extract useful information from them. This pipeline is important because weaknesses at any stage

reduce the value of logging. For example, a well-written log statement is less useful if it is lost, overwritten, or mixed with unstructured data that cannot be analyzed efficiently.

## 2.1 Instrumentation

Instrumentation is the preparation of a software system so that it can produce logs. In source code, this usually means inserting log statements that describe events or states. Instrumentation decisions include where to log, what variables to include, which verbosity level to choose, and how to configure the logging framework. These decisions determine whether logs will help developers debug failures during debugging.

The paper emphasizes that logging is not standardized across all systems. Developers log exceptions, return errors, assertions, progress information, and many other events. The difficulty is that too little logging hides relevant runtime context, while too much logging can create noise, performance overhead, and storage costs. Developers must therefore balance completeness with maintainability and efficiency.

## 2.2 Verbosity Levels and Configuration

Verbosity levels categorize log messages according to their purpose and severity. Common levels include `debug`, `info`, `warning`, `error`, and `critical` or `fatal`. Debug messages are detailed and intended mainly for development or troubleshooting. Info messages describe normal system behavior. Warnings indicate unusual situations that may become problems. Errors and critical messages describe failures that require attention. Choosing the wrong level can either hide important information or lead to too many unimportant messages.

Log configuration controls the behavior of logging without changing code. For example, a configuration file can define where logs are sent, how log entries are formatted, and which severity levels are recorded. This separation is useful because development and production environments often require different logging behavior. In production, debug-level messages might be disabled to reduce overhead, while warning and error messages remain enabled.

## 2.3 Storage

After execution, log data is commonly stored in files, databases, or dedicated log management systems. The amount of log data can be very large, which is why scalability is very important. Logs may also contain sensitive information such as

usernames, IP addresses, or business data. Therefore, storage solutions should support confidentiality, integrity, and availability.

The literature reviewed by Batoun et al. includes approaches such as log compression, immutable log storage, and pseudonymization. Compression reduces storage costs, immutable storage helps protect logs against tampering, and pseudonymization replaces sensitive information with pseudonyms so that logs remain useful without exposing personal data.

## 2.4 Analysis

Log analysis is difficult because raw logs are often large, noisy, and heterogeneous. Different systems and libraries produce different formats, and even the same application may produce multiple kinds of log messages. For this reason, many research papers focus on preprocessing steps such as log parsing, template identification, clustering, summarization, mining, and pattern extraction. These techniques convert raw textual log lines into more structured forms that can support later debugging, anomaly detection, or failure prediction.

For example, log parsing separates fixed text from variables. The raw log lines `User 42 failed login from 10.0.0.5` and `User 77 failed login from 10.0.0.8` can be generalized into the template `User <*> failed login from <*>`. This makes it easier to identify recurring event types. Such methods enable log mining, clustering, and automation for analysis.

## 3 Research Focus in the Literature

Batoun et al. classify the 204 reviewed papers according to the three main phases of the logging pipeline: instrumentation, storage, and analysis. The distribution shows that research attention is not equally spread across these phases. Most papers focus on log analysis, while instrumentation receives less attention and storage is covered by only a small part of the literature [1].

The largest part of the literature focuses on log analysis. This includes techniques that transform raw and often unstructured log data into a more useful form. Examples are log parsing, template identification, clustering, and mining. Specifically, researchers have studied how to improve both the performance and speed of these methods.

Instrumentation is the second largest research area. It concerns the creation of useful log statements. Typical questions are where to add log statements, which variables should be included, and which verbosity level should be used. One paper found

Table 1: Research focus across the logging pipeline.

| Pipeline phase  | Share of studied papers | Typical research topics                                                                      |
|-----------------|-------------------------|----------------------------------------------------------------------------------------------|
| Analysis        | 53%                     | Log parsing, template identification, clustering, summarization, mining, pattern extraction. |
| Instrumentation | 37%                     | Where to log, what to log, verbosity level, logging quality.                                 |
| Storage         | 10%                     | Compression, secure storage, pseudonymization, immutable storage.                            |

that 35% of log levels were inconsistent with the developer’s intention. In addition, automated tools are being developed to help developers implement logs correctly and detect bad logs.

Storage receives the least research attention in the reviewed literature. However, it is still important for practical logging because logs can become very large and may contain sensitive information. Research in this area discusses topics such as log compression, with up to a 94.98% compression rate, pseudonymization, and immutable log storage via blockchain-based methods.

Overall, the literature mainly focuses on making existing logs easier to process and analyze. This is valuable, but it also means that earlier phases of the pipeline, especially the creation and configuration of useful logs, receive less attention.

## 4 Developer Practice on Stack Overflow

To compare academic research with developer practice, Batoun et al. also analyze logging-related Stack Overflow questions. Their study includes 20,766 questions in the quantitative analysis and 149 questions in the qualitative analysis. From these questions, they identify seven high-level topics and 22 fine-grained topics [1].

The seven high-level topics are handling log files, dependency configuration of logging libraries, infrastructure-related configuration, usage of logging libraries in the code, scattered logging, parsing complex logging data, and context-dependent usage of logs. These topics show that many developer problems are practical configuration and usage problems rather than purely analytical problems.

A major topic is configuration, which 18,623 questions were about. Developers ask how to configure logging libraries, dependencies, formats, output destinations, and

infrastructure. These questions are important because logging can only support debugging if log messages are actually produced, sent to the correct place, and formatted in a usable way.

Another common topic is handling log files. Developers ask about log rotation, file names, overwriting existing logs, or managing several log outputs.

Stack Overflow questions also show problems with scattered and context-dependent logging. In many projects, log statements are distributed across several systems. This makes logs harder to search, compare, and analyze.

## 5 Research Gap

The comparison between the literature review and the Stack Overflow analysis shows a clear research–practice gap. In fact, 5 out of the 7 StackOverflow high-level topics were not covered by the literature.

While research often focuses on advanced forms of log analysis, developers frequently struggle with the earlier and more practical stages of logging. Many research papers treat logs mainly as data that can be parsed, clustered, mined, or analyzed after they have already been produced. Meanwhile, developers face challenges in previous steps of the logging pipeline, such as configuring logging frameworks, setting dependencies, and choosing output destinations. In addition, they often have challenges with operational issues, like rotating log files correctly, ensuring that log files are not being overwritten, and dealing with logs from different sources.

For debugging, this gap is especially relevant. Logs are supposed to provide evidence about runtime behavior, but they can only do so if they contain the right information, are configured correctly, and remain accessible when failures occur. A complete logging strategy should therefore connect instrumentation, storage, and analysis instead of treating them as separate problems.

## 6 Critical Reflection

The paper provides a valuable overview, but several limitations should be considered. First, Stack Overflow does not represent all developer practice. It is heavily influenced by its userbase. Beginners may be overrepresented, while industrial logging problems may be underrepresented as they are solved with internal tools. As a result, the identified research–practice gap may be partly shaped by the data source.

In addition, the research focus on analysis techniques is not necessarily wrong. Large-



scale log analysis is a real problem, especially for distributed systems and cloud infrastructure. Meanwhile, operational problems can often be solved using documentation or forums and do not require further research.

## 7 Implications

### 7.1 Implications for Developers

The paper recommends that developers should use existing research and tools for log instrumentation so that enough information for debugging is captured while avoiding performance and storage overhead.

In addition, they should review existing logs periodically, using guidelines so that they can fix problems before they grow. They should also adopt automated log-analysis techniques instead of relying solely on manual parsing.

### 7.2 Implications for Researchers

Researchers should continue improving automated log analysis, but the paper indicates a need for more research on practical configuration and usage problems. Moreover, logging infrastructure needs more attention according to the paper.

Another important research direction is heterogeneous logging from different sources, especially minimizing inconsistencies between logging-library versions.

## 8 Conclusion

Software logging is useful because it records runtime context that developers and operators cannot otherwise observe. However, logging is difficult because it requires careful decisions about what to log, where to log, which level to use, how to store logs, and how to analyze them. The paper by Batoun et al. (2024) provides a broad overview of these challenges by combining a systematic literature review with an analysis of developer questions.

The main result is that research focuses mostly on log analysis, while developers often struggle with configuration and everyday usage problems. This mismatch is important for debugging. Advanced analysis tools are valuable, but they depend on correctly produced and managed logs. A complete logging strategy must therefore connect instrumentation, storage, and analysis. For practitioners, the key lesson is to make

logs consistent, secure, and maintainable. For researchers, the key lesson is to address the practical problems that developers face when trying to make logging work in real systems.

## Bibliography

- [1] Batoun, M. A., Sayagh, M., Aghili, R., Ouni, A., and Li, H. (2024). A literature review and existing challenges on software logging practices: From the creation to the analysis of software logs. *Empirical Software Engineering*, 29, Article 103. <https://doi.org/10.1007/s10664-024-10452-w>
- [2] Zeller, A. (2009). *Why Programs Fail: A Guide to Systematic Debugging* (2nd ed.). Morgan Kaufmann.
- [3] Chen, B., and Jiang, Z. M. (2021). A survey of software log instrumentation. *ACM Computing Surveys*, 54(4), 1–34. <https://doi.org/10.1145/3448976>
- [4] El-Masri, D., Petrillo, F., Guéhéneuc, Y.-G., Hamou-Lhadj, A., and Bouziane, A. (2020). A systematic literature review on automated log abstraction techniques. *Information and Software Technology*, 122, 106276. <https://doi.org/10.1016/j.infsof.2020.106276>
- [5] Mendes, E., and Petrillo, F. (2021). Log severity levels matter: A multivocal mapping. In *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*, 1002–1013. <https://doi.org/10.1109/QRS54544.2021.00109>
- [6] Rong, G., Zhang, Q., Liu, X., and Gu, S. (2017). A systematic review of logging practice in software engineering. In *2017 24th Asia-Pacific Software Engineering Conference (APSEC)*, 534–539. <https://doi.org/10.1109/APSEC.2017.61>